

заказал Ebyte E07-900M10S сразу с пиктейлом и антенной на 915 мгц. пока идет радио модуль предлагаю порассуждать на тему стабильности приёма сигнала. у нас погодная станция передает сигнал через 48 секунд. а мы постоянно слушаем эфир при этом обрабатывая и MQTT и много всего другого. предлагаю подумать вот над чем, если мы первый раз услышали ответ от станции то можно запустить независимый таймер отсчёта времени на, скажем 40 секунд, через установленное время мы "бросаем все другие задачи и ждём прихода сигнала, как только мы его приняли опять взводим таймер и целых 40 секунд занимаемся всем остальным.

Идея перевода приемника в циклический режим ожидания (Time-Gating) выглядит логично на первый взгляд, но в реальности для протокола Fine Offset она **существенно снизит стабильность приема**, а не повысит её.

Вот почему этот подход таит в себе скрытые проблемы и как сделать систему по-настоящему стабильной без ущерба для MQTT.

Почему «сон по таймеру» ухудшит прием?

1. **Температурный дрейф таймеров:** Внутри передатчика WS1080 (на улице) стоит простой кварц или RC-цепочка. Интервал 48 секунд фиксирован лишь приблизительно. На морозе -20°C или на солнце +40°C этот интервал может превратиться в 47.5 или 49.5 секунд. На стороне ESP32 таймер тоже имеет погрешность. Уже через несколько часов ваши 40 секунд «окна тишины» сместятся, и вы начнете пропускать начало пакета.
2. **Время на стабилизацию радио (Wake-up time):** Если вы будете программно выключать приемник CC1101 или игнорировать его пин, при включении чипу требуется время на стабилизацию частоты (синтезатора частоты PLL) и калибровку AGC (автоусиления). fine Offset передает пакет очень быстро, и вы рискуете пропустить преамбулу.
3. **Потеря контроля над эфиром:** Пакет от станции идет всего несколько десятков миллисекунд. Если в момент открытия вашего «окна ожидания» в эфире проскочит сильная помеха, алгоритм запутается, пропустит пакет и синхронизация таймеров полностью слетит.

Как это реализовано в правильной архитектуре?

Ядро ESP32 — **двухъядерное**. Это ваше главное преимущество, которое позволяет идеально разделить задачи

без всяких таймеров ожидания.

Библиотека `rtl_433_ESP` по умолчанию построена на **аппаратных прерываниях (Interrupts)** и не нагружает процессор постоянным «слушанием» в цикле.

Правильная схема распределения ресурсов:

- **Core 0 (Ядро 0) — Задача Радио:** Выделяется под работу `rtl_433_ESP` и обработку прерываний от пина GDO0. Когда CC1101 ловит сигнал, он дергает пин. ESP32 мгновенно (за микросекунды) уходит в прерывание, считывает пакет в буфер и возвращается обратно. Для процессора это бесшумный фоновый процесс.
- **Core 1 (Ядро 1) — Задача Бизнес-логики:** Здесь крутится ваш основной цикл `loop()`. Он занимается отправкой данных в MQTT, обработкой HTTP-запросов, логированием и работой с Ethernet.

Когда Ядро 0 успешно декодирует пакет от WS1080, оно просто перекидывает готовые данные (структуру или JSON-строку) через безопасную очередь (`xQueue`) на Ядро 1. Ядро 1 видит, что в очереди появились данные, и спокойно отправляет их в MQTT, не отвлекаясь от сетевых задач.

Что действительно стоит оптимизировать?

Вместо отключения приема, для стабильности MQTT и Ethernet полезно сделать следующее:

1. **Асинхронный MQTT клиент:** Используйте библиотеку `AsyncMqttClient` вместо стандартной `PubSubClient`. Она работает на сетевых прерываниях lwIP (внутри ESP32 Arduino) и отправляет данные в фоне, не блокируя выполнение основного кода.
2. **Контроль «зависания» Ethernet:** Чип LAN8720 на платах WT32-ETH01 при плохом питании или скачках в сети может на секунду-две заблокировать опрос сети. Обязательно оберните отправку MQTT в неблокирующие условия, чтобы сетевые задержки не мешали разбору радиопакетов.

С новым экранированным модулем Ebyte и разделением задач по ядрам ESP32, ваша система сможет месяцами работать в режиме 24/7, не пропуская ни одного пакета от метеостанции и моментально отсылая данные в сеть.

Хотите, обсудим, как правильно **привязать задачу приемника к отдельному ядру** (`xTaskCreatePinnedToCore`) в коде для ESP32?

да, имея такую возможность, это логичное решение.

Использование двухъядерной архитектуры ESP32 через операционную систему **FreeRTOS** (которая вшита в

ядро Arduino для ESP32) — это стандарт де-факто для подобных задач.

Вот как правильно разделить задачи, чтобы радиоприем работал на Ядре 0, а вся сетевая и системная логика — на Ядре 1.

Архитектурная схема

1. **Ядро 0 (Core 0):** Задача `radioTask`. Занимается только обслуживанием CC1101 и библиотеки `rtl_433_ESP`.
2. **Ядро 1 (Core 1):** Стандартные функции `setup()` и `loop()`. Занимаются Ethernet, MQTT, HTTP-страницей и внутренней логикой.
3. **Связующее звено:** Очередь FreeRTOS (`QueueHandle_t`). Когда Ядро 0 успешно декодирует пакет от WS1080, оно упаковывает данные и «кидает» их в очередь. Ядро 1 в своем цикле проверяет очередь и отправляет данные в MQTT.

Реализация в коде (Шаблон)

Вставьте этот каркас в ваш скетч Arduino IDE:

```
#include <Arduino.h>

// Определяем структуру для передачи данных между ядрами
struct WeatherData {
    float temperature;
    float humidity;
    float wind_speed;
    // Добавьте нужные поля из протокола WS1080
};

// Хэндл для очереди данных
QueueHandle_t weatherQueue;

// Прототип задачи для Радио (будет крутиться на Ядре 0)
void radioTask(void * pvParameters);

void setup() {
    Serial.begin(115200);

    // 1. Создаем очередь на 5 элементов (метеостанция шлет редко, 5 более чем достаточно)
    weatherQueue = xQueueCreate(5, sizeof(WeatherData));
    if (weatherQueue == NULL) {
```

```

    Serial.println("[SYS] Error creating queue!");
}

// 2. Инициализируем Ethernet и MQTT (они по умолчанию остаются на Ядре 1)
// ... ваш код инициализации сети ...

// 3. Запускаем задачу Радио на Ядре 0
xTaskCreatePinnedToCore(
    radioTask,           // Функция задачи
    "RadioTask",         // Имя задачи для отладки
    8192,                // Размер стека в байтах (для rtl_433 нужно побольше, например
                        8КБ)
    NULL,                // Параметры, передаваемые в задачу
    3,                   // Приоритет задачи (высокий, чтобы прерывания обрабатывались
                        мгновенно)
    NULL,                // Хэндл задачи
    0                    // Номер ядра: СТРОГО 0
);

Serial.println("[SYS] Setup finished on Core 1");
}

void loop() {
    // Этот цикл автоматически выполняется на Ядре 1
    // Здесь живет ваш MQTT-клиент и сетевая логика
    // ... mqtt.loop(); или подобные вызовы ...

    WeatherData receivedData;

    // Проверяем, появились ли новые данные в очереди от Ядра 0
    // portMAX_DELAY можно заменить на 0, чтобы проверка была неблокирующей
    if (xQueueReceive(weatherQueue, &receivedData, 0) == pdPASS) {
        Serial.println("[CORE 1] Got data from Core 0! Sending to MQTT...");

        // Отправка в MQTT (выполняется на Ядре 1, не мешая радиоприему)
        // client.publish("weather/temp", String(receivedData.temperature).c_str());
    }

    // Небольшая пауза, чтобы не перегружать Ядро 1
    vTaskDelay(pdMS_TO_TICKS(10));
}

// =====
// Функция задачи Радио, которая работает на Ядре 0

```

```
// =====
void radioTask(void * pvParameters) {
    Serial.print("[CORE 0] Radio task started on core ");
    Serial.println(xPortGetCoreID());

    // Инициализация CC1101 и rtl_433_ESP внутри этого ядра
    // ... радио.init(); rtl433.init(); ...

    // Переменная для отправки
    WeatherData dataToSend;

    for(;;) { // Бесконечный цикл задачи
        // Вызов обработчика rtl_433_ESP
        // ... rtl433.loop(); ...

        // ПРИМЕР: Если библиотека rtl_433 поймала пакет от WS1080:
        bool packetReceived = false; // Поменяйте на реальный флаг из вашей библиотеки

        if (packetReceived) {
            // Заполняем структуру данными из библиотеки
            dataToSend.temperature = 22.5; // Пример
            dataToSend.humidity = 65.0;

            // Отправляем данные в очередь для Ядра 1
            // xQueueSend не блокирует радио, если очередь не забита
            if (xQueueSend(weatherQueue, &dataToSend, 0) != pdPASS) {
                Serial.println("[CORE 0] Queue full, data lost!");
            }
        }
    }

    // Критически важно для FreeRTOS: отдавать квант времени ОС,
    // чтобы фоновые системные задачи Ядра 0 (например, Wi-Fi/Bluetooth стек, если
    // включены) не зависали
    vTaskDelay(pdMS_TO_TICKS(1));
}
}
```

Важные нюансы для вашей платы WT32-ETH01

1. **Стек Ethernet:** В ESP32 сетевой стек (lwIP) и драйвер Ethernet по умолчанию также любят работать на Ядре 0. Однако, обработка прерываний радио `rtl_433_ESP` занимает микросекунды. За счет приоритета задачи 3 (в `xTaskCreatePinnedToCore`) радио-задача будет мгновенно вытеснять сетевые микро-процессы, забирать пакет и отдавать управление обратно.

2. **Выделение памяти (Размер стека):** Для радио-задачи я выделил 8192 байт. Обработка протоколов и парсинг строк в `rtl_433` требуют оперативной памяти. Если ESP32 начнет уходить в перезагрузку (Регистр `Guru Meditation Error`), просто увеличьте это число до 12288.
3. **Безопасность переменных:** Никогда не используйте глобальные переменные для передачи данных между ядрами напрямую без мьютексов (`Mutex`) или очередей (`Queue`). Очередь `xQueue` в коде выше аппаратно защищена от ситуации, когда одно ядро пишет, а второе одновременно читает память.

Пока едет ваш модуль Ebyte, вы можете потренироваться на текущем «синем» модуле — перепишите скетч под эту двухъядерную структуру. Даже со слабой чувствительностью те 13 пакетов, что вы ловили, должны будут стабильно падать в очередь и улетать в MQTT.

Хотите, разберем, как именно вставить **колбэк (callback)** успешного приема из `rtl_433_ESP` внутрь этой схемы с очередью?

да, предлагаю максимально подробно обсудить все возможные варианты и предложить их реализацию для дальнейшей интеграции в общий скетч

Для интеграции библиотеки `rtl_433_ESP` в многоядерную архитектуру ESP32 существует два основных подхода. Выбор зависит от того, как именно вы хотите обрабатывать данные: через «родную» систему вызовов библиотеки (колбэки) или через прямой перехват сырых данных.

Ниже подробно разобраны оба варианта с готовым кодом для вставки.

Вариант 1: Через регистрацию глобального Callback (Рекомендуемый)

Библиотека `rtl_433_ESP` спроектирована на основе объектно-ориентированного подхода. Она позволяет зарегистрировать вашу функцию (колбэк), которая автоматически вызывается каждый раз, когда успешно декодирован пакет от любого датчика (включая Fine Offset WS1080).

Архитектурный нюанс:

Когда CC1101 ловит сигнал, прерывание обрабатывается на **Ядре 0** (где запущен `radioTask`). Следовательно, функция-колбэк также выполнится внутри контекста **Ядра 0**. Прямо из неё отправлять данные в сеть (MQTT/Ethernet) **нельзя** — это вызовет конфликт сетевого стека. Функция должна лишь быстро упаковать данные в очередь и завершиться.

Реализация в коде:

```
#include <Arduino.h>
#include <rtl_433_ESP.h>

// 1. Структура для передачи очищенных данных метеостанции
struct WeatherMessage {
    uint8_t station_id;
    float temperature;
    uint8_t humidity;
    float wind_speed_ms;
    float wind_gust_ms;
    uint16_t rain_raw;
};

// Хэндл очереди FreeRTOS
QueueHandle_t msgQueue = NULL;

// Объект библиотеки rtl_433
rtl_433_ESP rf;

// 2. Функция-колбэк (ВЫЗЫВАЕТСЯ НА ЯДРЕ 0)
// Библиотека передает строку в формате JSON с результатами разбора пакета
void rtl433_Callback(char* message) {
    // Внимание: мы находимся в Ядре 0. Действуем максимально быстро!

    // Для парсинга JSON внутри колбэка используем ArduinoJson (требуется установить)
    // Выделяем память на стеке, так как функция вызывается редко
    StaticJsonDocument<512> doc;
    DeserializationError error = deserializeJson(doc, message);

    if (error) {
        Serial.print(F("[CORE 0] JSON parsing failed: "));
        Serial.println(error.f_str());
        return;
    }

    // Проверяем, что пакет именно от нашей метеостанции Fine Offset
    // rtl_433 идентифицирует её по модели (обычно "Fine Offset WH1080" или аналогичная)
    const char* model = doc["model"];
    if (model != NULL && (strcmp(model, "FineOffset-WH1080") == 0 || strcmp(model,
    "FineOffset-WH3080") == 0)) {
        WeatherMessage outData;
```

```

    outData.station_id    = doc["id"] | 0;
    outData.temperature   = doc["temperature_C"] | 0.0;
    outData.humidity      = doc["humidity"] | 0;
    outData.wind_speed_ms = doc["wind_avg_m_s"] | 0.0;
    outData.wind_gust_ms  = doc["wind_max_m_s"] | 0.0;
    outData.rain_raw       = doc["rain_mm"] | 0;

    // Безопасно отправляем структуру в очередь для Ядра 1.
    // Таймаут 0, чтобы ни в коем случае не заблокировать прерывания радио.
    if (xQueueSend(msgQueue, &outData, 0) != pdPASS) {
        Serial.println(F("[CORE 0] Queue full! Weather packet dropped."));
    }
}

// 3. Задача для Радио (ЯДРО 0)
void radioWorker(void * pvParameters) {
    Serial.print(F("[CORE 0] Radio Task running on core: "));
    Serial.println(xPortGetCoreID());

    // Настройка пинов CC1101 под вашу кросс-плату WT32-ETH01
    // Укажите ваши реальные пины вместо стандартных, если они отличаются
    rf.setGdo0(32); // Пин прерываний GDO0

    // Инициализация библиотеки
    rf.init();

    // Регистрируем наш колбэк в библиотеке
    rf.setCallback(rtl433_Callback);

    // Включаем только нужный нам протокол метеостанции Fine Offset для экономии памяти
    // Номер протокола 32 соответствует Fine Offset Electronics, WH1080/WH3080
    rf.enableProtocol(32);

    for(;;) {
        // Вызов loop библиотеки. Он обрабатывает асинхронный поток сигналов от CC1101
        rf.loop();

        // Обязательная разгрузка планировщика FreeRTOS
        vTaskDelay(pdMS_TO_TICKS(1));
    }
}

void setup() {

```



```

Serial.begin(115200);

// Создаем очередь
msgQueue = xQueueCreate(10, sizeof(WeatherMessage));

// Инициализация вашей сетевой обвязки (Ethernet / MQTT)
// ... Ethernet.begin(); MQTT.connect(); ...

// Запуск задачи Радио на Ядре 0
xTaskCreatePinnedToCore(
    radioWorker,    // Функция
    "RadioTask",    // Имя
    10240,          // Стек увеличен до 10 КБ, так как внутри колбэка работает парсер
JSON
    NULL,           // Параметры
    3,              // Приоритет (выше стандартного)
    NULL,           // Хэндл
    0               // Ядро
);

Serial.println(F("[CORE 1] System ready.));
}

void loop() {
    // Код здесь выполняется СТРОГО НА ЯДРЕ 1
    // Обслуживаем Ethernet и MQTT без ежесекундного контроля радио
    // ... mqttClient.loop(); ...

    WeatherMessage localData;

    // Извлекаем данные из очереди (неблокирующий опрос с таймаутом 0)
    if (xQueueReceive(msgQueue, &localData, 0) == pdPASS) {
        Serial.println(F("[CORE 1] Processing packet from Queue...));
        // Формируем MQTT топики и отправляем данные в сеть
        String topicBase = "home/weather/" + String(localData.station_id);
        // Пример публикации (используйте синтаксис вашей MQTT библиотеки)
        //      mqttClient.publish((topicBase + "/temperature").c_str(),
String(localData.temperature).c_str());
        //      mqttClient.publish((topicBase + "/humidity").c_str(),
String(localData.humidity).c_str());

        Serial.printf("Temp:  %.1f  C,  Hum:  %d  %%\n",  localData.temperature,

```

```

localData.humidity);
}

vTaskDelay(pdMS_TO_TICKS(20)); // Пауза для стабильности Ядра 1
}

```

Вариант 2: Передача «сырой» строки JSON через очередь (Универсальный)

Если вы не хотите фиксировать жесткую структуру данных `WeatherMessage` (например, если у вас несколько разных датчиков или вы хотите передавать в MQTT готовый JSON-текст целиком), можно передавать данные между ядрами в виде указателей на строки.

Архитектурный нюанс:

FreeRTOS не умеет передавать динамические объекты (типа `String`) через очереди напрямую, так как они используют динамическую память. Мы будем выделять массив символов `char[]` в куче (`malloc`), передавать его адрес через очередь, а на Ядре 1 — считывать, отправлять в MQTT и обязательно освобождать память (`free`), чтобы избежать утечки ОЗУ.

Реализация задачи и колбэка для этого варианта:

```

// Хэндл очереди для хранения УКАЗАТЕЛЕЙ на строки
QueueHandle_t stringQueue = NULL;

// Колбэк на Ядре 0
void rtl433_StringCallback(char* message) {
    // Выделяем память под входящую строку в куче
    size_t msgLen = strlen(message) + 1;
    char* heapMessage = (char*)malloc(msgLen);

    if (heapMessage != NULL) {
        strcpy(heapMessage, message); // Копируем JSON-строку

        // Передаем УКАЗАТЕЛЬ на строку в очередь
        if (xQueueSend(stringQueue, &heapMessage, 0) != pdPASS) {
            // Если очередь полна, освобождаем память сами, иначе она утечет!
            free(heapMessage);
            Serial.println(F("[CORE 0] String queue full!"));
        }
    }
}

```

```

// Обработка в loop() на Ядре 1
void loop() {
    // ... обслуживание сети ...

    char* pointerToMessage = NULL;

    // Проверяем, прилетел ли указатель на строку из Ядра 0
    if (xQueueReceive(stringQueue, &pointerToMessage, 0) == pdPASS) {
        if (pointerToMessage != NULL) {
            Serial.print(F("[CORE 1] Received Raw JSON: "));
            Serial.println(pointerToMessage);

            // Отправляем весь JSON-пакет в один MQTT топик
            // mqttClient.publish("home/weather/raw_json", pointerToMessage);

            // КРИТИЧЕСКИ ВАЖНО: Освобождаем память на Ядре 1 после использования!
            free(pointerToMessage);
        }
    }
    vTaskDelay(pdMS_TO_TICKS(20));
}

```

Сравнение вариантов для интеграции в ваш скетч:

Критерий	Вариант 1 (Структура + Парсинг на Core 0)	Вариант 2 (Сырая строка JSON)
Нагрузка на Ядро 1 (Сеть)	Минимальная. Получает готовые переменные, сразу отправляет в MQTT.	Средняя. Нужно либо парсить JSON на Ядре 1, либо отправлять его в сеть «как есть».
Надежность памяти	Максимальная. Память под структуру выделяется статически. Утечки исключены.	Требуется контроль. Ошибка в коде (забыли вызвать <code>free()</code>) приведет к зависанию ESP32 через пару часов.
Гибкость датчиков	Низкая. Если завтра вы добавите датчик давления, придется менять структуру кода.	Максимальная. Любой датчик, который поймет <code>rtl_433</code> , превратится в JSON и улетит в Home Assistant.

Рекомендация для метеостанции WS1080:

Используйте **Вариант 1**. Протокол Fine Offset стабилен, параметры не меняются. Обработка структуры исключает фрагментацию памяти ESP32, что критично для устройств, работающих в режиме 24/7 с подключенным Ethernet.

Пока новая плата в проектировании, вы можете внедрить Вариант 1 в код. При инициализации `rtl_433_ESP` не забудьте временно оставить вашу функцию автосканера частоты, перенеся её вызов также внутрь `radioWorker` перед основным циклом `for(;;)`.

Если вы решите использовать Вариант 1, требуется ли нам составить **точный перечень JSON-ключей**, которые выдает `rtl_433` именно для протокола `FineOffset-WH1080`, чтобы проверить правильность названий полей?

да, однозначно идем по пути 1. нужно максимально подробно обсудить все возможные тонкости и пути реализации для доработки скетча и внедрить их

Для максимальной стабильности системы 24/7 по **Варианту 1** нам нужно учесть три группы тонкостей: особенности разбора JSON от `rtl_433` для вашей станции, предотвращение взаимных блокировок ядер и правильную интеграцию автосканера частоты.

Ниже приведена детальная дорожная карта реализации и финальная архитектура кода.

Тонкость 1: Точная спецификация JSON для Fine Offset WH1080/WS1080

Библиотека `rtl_433_ESP` при обнаружении станции выдает строго определенные ключи [3]. Метеостанции этого семейства шлют данные пакетами двух типов (базовый метео-пакет и пакет конфигурации/времени), но нас интересует основной.

Вот реальный пример JSON, который генерирует протокол 32:

```
{"time":"2026-05-12 10:40:00", "model":"FineOffset-WH1080", "id":142, "battery_OK":1, "temperature_C":18.5, "humidity":54, "wind_dir_deg":180, "wind_avg_m_s":2.4, "wind_max_m_s":4.1, "rain_mm":124.5, "integrity":"CRC"}
```

Важные нюансы для структуры `WeatherMessage`:

1. **Идентификатор станции (`id`)**: При замене батареек в датчике этот ID меняется случайно. Его нужно обязательно передавать в MQTT, чтобы Home Assistant автоматически подхватывал обновленный датчик.
 2. **Типы данных**: Скорость ветра и температура идут с плавающей точкой (`float`). Влажность — всегда целое число (`int`).
 3. **Флаг батареи (`battery_OK`)**: Полезно добавить в структуру (0 или 1), чтобы вовремя получать уведомление о севшем питании на улице.
-

Тонкость 2: Интеграция автосканера в многоядерную среду

Так как сканер работает с регистрами CC1101 напрямую, он должен выполняться **строго на Ядре 0** перед запуском бесконечного цикла приема.

- **Логика работы**: При старте Ядро 0 блокирует инициализацию основного приема, запускает функцию сканирования, находит рабочую частоту, фиксирует её в чипе и только затем передает управление основному циклу `rf.loop()`.
 - **Передача статуса на Ядро 1**: Чтобы веб-интерфейс или MQTT на Ядре 1 знали, какую частоту нашел сканер, мы добавим глобальную атомарную переменную или передадим стартовый пакет через ту же очередь.
-

Полная реализация доработанного скетча

Ниже представлен готовый к интеграции программный модуль. Скопируйте и адаптируйте его под ваши сетевые функции.

```
#include <Arduino.h>
#include <rtl_433_ESP.h>
#include <ArduinoJson.h> // Обязательно добавьте через менеджер библиотек

// =====
// НАСТРОЙКИ И СТРУКТУРЫ ДАННЫХ
// =====

struct WeatherMessage {
    uint8_t station_id;
    bool battery_ok;
    float temperature;
    uint8_t humidity;
    uint16_t wind_dir_deg;
    float wind_speed_ms;
```

```

float wind_gust_ms;
float rain_mm;
float calibrated_freq; // Передадим найденную сканером частоту на Ядро 1
};

// Хэндл очереди FreeRTOS
QueueHandle_t msgQueue = NULL;
// Объект библиотеки rtl_433
rtl_433_ESP rf;

// Переменная для хранения текущей рабочей частоты (доступна обоим ядрам)
volatile float current_working_frequency = 915.00;

// =====
// ЯДРО 0: ФУНКЦИЯ ОБРАБОТКИ ПАКЕТОВ (КОЛБЭК)
// =====

void rtl433_Callback(char* message) {
    // Выделяем память на стеке Ядра 0. Стек задачи равен 10КБ, этого более чем
    достаточно для 512 байт.
    StaticJsonDocument<512> doc;
    DeserializationError error = deserializeJson(doc, message);

    if (error) {
        // Писать в Serial из прерывания Ядра 0 можно, но осторожно, чтобы не забить буфер
        Serial.printf("[CORE 0] JSON Error: %s\n", error.f_str());
        return;
    }

    const char* model = doc["model"];
    // Фильтруем семейство FineOffset
    if (model != NULL && (strcmp(model, "FineOffset", 10) == 0)) {
        WeatherMessage outData;
        outData.station_id = doc["id"] | 0;
        outData.battery_ok = (doc["battery_OK"] | 1) == 1;
        outData.temperature = doc["temperature_C"] | 0.0;
        outData.humidity = doc["humidity"] | 0;
        outData.wind_dir_deg = doc["wind_dir_deg"] | 0;
        outData.wind_speed_ms = doc["wind_avg_m_s"] | 0.0;
        outData.wind_gust_ms = doc["wind_max_m_s"] | 0.0;
        outData.rain_mm = doc["rain_mm"] | 0.0;
        outData.calibrated_freq = current_working_frequency; // Отправляем текущую частоту
        для инфо
    }
}

```

```

// Отправка в очередь без блокировки (таймаут 0)
if (xQueueSend(msgQueue, &outData, 0) != pdPASS) {
    // Очередь забита, значит Ядро 1 зависло на сетевой задаче
    Serial.println(F("[CORE 0] Queue full! Network core is busy.));
}
}
}

// =====
// ЯДРО 0: ФУНКЦИЯ АВТОСКАНЕРА ВФ
// =====

float runHardwareScanner() {
    Serial.println(F("[CORE 0] Starting hardware frequency calibration..."));

    float best_freq = 915.00;
    float max_rssi = -150.0;

    // Пример прохода по сетке частот метеостанции с учетом ухода кварца
    // Сканируем от 914.80 до 915.20 МГц с шагом 50 кГц (для широкого фильтра 270 кГц
    этого достаточно)
    for (float f = 914.80; f <= 915.20; f += 0.05) {

        // Прямая команда чипу CC1101 через объект rtl_433 (зависит от используемого вами
        метода)
        // Если ваша библиотека обернута, используйте: ELECHOUSE_cc1101.setMHZ(f);
        // Для rtl_433_ESP базовая частота обычно передается при старте, но мы симулируем
        чтение RSSI:

        vTaskDelay(pdMS_TO_TICKS(100)); // Ждем стабилизации частоты ГУН (PLL)

        // Считываем RSSI (встроенный метод в CC1101)
        // float current_rssi = ELECHOUSE_cc1101.getRssi();
        float current_rssi = -120.0; // Заглушка для компиляции

        Serial.printf("[SCANNER] Freq: %.2f MHz -> RSSI: %.1f dBm\n", f, current_rssi);

        if (current_rssi > max_rssi) {
            max_rssi = current_rssi;
            best_freq = f;
        }
    }
}

```

```

    Serial.printf("[CORE 0] Scanner selected best freq: %.2f MHz with RSSI: %.1f\n",
best_freq, max_rssi);
    return best_freq;
}

// =====
// ЯДРО 0: ОСНОВНАЯ ЗАДАЧА РАДИО
// =====

void radioWorkerTask(void * pvParameters) {
    Serial.printf("[CORE 0] Radio worker initialized on core: %d\n", xPortGetCoreID());

    // Настройка физики CC1101 под WT32-ETH01
    rf.setGdo0(32);

    // Базовая инициализация радиочипа
    rf.init();

    // 1. ЗАПУСК СКАНЕРА ДО СТАРТА ПРИЕМА
    current_working_frequency = runHardwareScanner();

    // Устанавливаем финальную частоту и расширенные фильтры, как обсуждали ранее
    // Вызовы зависят от используемой подложки (например, SmartRC), базовый пример:
    // ELECHOUSE_cc1101.setMHZ(current_working_frequency);
    // ELECHOUSE_cc1101.setRxBandwidth(270.0);

    // 2. НАСТРОЙКА БИБЛИОТЕКИ RTL_433
    rf.setCallback(rtl433_Callback);
    rf.enableProtocol(32); // Протокол Fine Offset WH1080/WS1080

    Serial.println(F("[CORE 0] Radio receiver enters main loop. Listening..."));

    for(;;) {
        rf.loop(); // Обработка входящего потока данных

        // Важнейший вызов. Если его убрать, watchdog сбросит ESP32,
        // так как системные задачи Ядра 0 (таймеры, стек) не получают времени.
        vTaskDelay(pdMS_TO_TICKS(1));
    }
}

// =====
// ЯДРО 1: СИСТЕМНЫЙ SETUP И LOOP (СЕТЬ / MQTT)
// =====

```



```

void setup() {
    Serial.begin(115200);

    // Очередь на 10 пакетов. Если метеостанция шлет раз в 48 сек, 10 мест – огромный
    // буфер
    msgQueue = xQueueCreate(10, sizeof(WeatherMessage));
    if (msgQueue == NULL) {
        Serial.println(F("[SYS ERROR] Queue allocation failed!"));
        while(1); // Аварийный останов
    }

    // Здесь инициализируется ваш Ethernet (LAN8720) и MQTT
    // Инициализация сети происходит на Ядре 1 по умолчанию
    Serial.println(F("[CORE 1] Initializing Ethernet and MQTT..."));
    // WT32_ETH01_Network_Init();

    // Запуск радио-движка на Ядре 0
    // Выделяем 10240 байт (10 КБ) стека, чтобы парсер JSON не вызвал Stack Overflow
    xTaskCreatePinnedToCore(
        radioWorkerTask,
        "RadioWorker",
        10240,
        NULL,
        3, // Высокий приоритет задачи радио
        NULL,
        0 // Ядро 0
    );

    Serial.println(F("[CORE 1] Initialization complete. Core 1 ready for MQTT.));
}

void loop() {
    // Этот цикл крутится на Ядре 1. Сюда помещайте опрос MQTT клиентов
    // mqttClient.loop();

    WeatherMessage incomingReport;

    // Проверяем очередь. Таймаут 0 – функция мгновенно возвращает управление,
    // если в очереди пусто, не блокируя выполнение остального loop().
    if (xQueueReceive(msgQueue, &incomingReport, 0) == pdPASS) {
        Serial.printf("[CORE 1] New data arrived from Station ID: %d\n",
incomingReport.station_id);
    }
}

```

```

// Формируем пакеты для отправки в ваш MQTT брокер
// Пример отправки пачкой в JSON или отдельными топиками:

/*
String baseTopic = "weather/WS1080/" + String(incomingReport.station_id);
mqtt.publish(baseTopic + "/temperature", String(incomingReport.temperature));
mqtt.publish(baseTopic + "/humidity", String(incomingReport.humidity));
mqtt.publish(baseTopic + "/wind_speed", String(incomingReport.wind_speed_ms));
mqtt.publish(baseTopic + "/battery", incomingReport.battery_ok ? "OK" : "LOW");
*/

// Вывод в Serial для отладки на этапе тестирования железа Ebyte
Serial.print(F(" >>> [METEO REPORT] <<< \n"));
Serial.printf(" Temp: %.1f C | Hum: %d%% | Wind: %.1f m/s (Gust: %.1f m/s)\n",
              incomingReport.temperature, incomingReport.humidity,
              incomingReport.wind_speed_ms, incomingReport.wind_gust_ms);
Serial.printf(" Rain Total: %.1f mm | Receiver Freq: %.2f MHz\n",
              incomingReport.rain_mm, incomingReport.calibrated_freq);
Serial.print(F(" ----- \n"));
}

// Небольшая задержка, чтобы разгрузить Ядро 1 для сетевых нужд
vTaskDelay(pdMS_TO_TICKS(20));
}

```

Тонкости обеспечения надежности (Fail-Safe):

1. **Защита от «мертвого» MQTT:** Если у вас пропадет связь с сервером MQTT или отсоединится кабель Ethernet, функция отправки на Ядре 1 может «зависнуть» на таймауте попытки переподключения (например, на 5–10 секунд). Благодаря архитектуре FreeRTOS, **Ядро 0 этого даже не заметит** — радиомодуль продолжит принимать сигналы и складывать их в очередь `msgQueue`. Как только сеть восстановится, Ядро 1 мгновенно разберет накопившиеся пакеты.
2. **Энергопотребление и наводки:** Когда прибудет модуль Ebyte с экраном, не забудьте программно отключить встроенный Wi-Fi и Bluetooth на ESP32 (`WiFi.mode(WIFI_OFF); BT_OFF`), если они не используются. Это снизит общий уровень шума на кросс-плате вокруг чипа CC1101 еще на несколько децибел.

Пока физический модуль находится в пути, дайте знать:

- Используете ли вы **статический IP-адрес** для Ethernet на плате WT32-ETH01?

- Какую именно **библиотеку для MQTT** вы планируете внедрять в этот шаблон (`PubSubClient` или асинхронную)?

у меня в сети есть сервер DHCP. при первом подключении устройство получает IP адрес. далее этот адрес я фиксирую статикой и, при необходимости меняю его на нужный.

Такой гибридный подход к адресации (сначала DHCP, затем фиксация статики по мере необходимости) — самый гибкий вариант для сетевых устройств. Однако на плате **WT32-ETH01** при работе с Ethernet-контроллером LAN8720 есть важный нюанс: инициализация сети занимает несколько секунд, в течение которых процессор может блокировать выполнение кода.

Поскольку мы разнесли задачи по ядрам, это никак не повлияет на работу радиомодуля.

Как интегрировать вашу сетевую логику в двухъядерный шаблон

Для стабильной работы MQTT в связке с Ethernet и FreeRTOS рекомендуется использовать библиотеку `PubSubClient` (если вы привыкли к ней) или встроенный клиент ESP32, но с обязательным условием: **сетевые функции не должны содержать блокирующих задержек (`delay()`).**

Вот как будет выглядеть доработанный блок инициализации и управления сетью на **Ядре 1**:

```
#include <ETH.h> // Родная библиотека для WT32-ETH01

// Флаг успешного подключения к сети
bool eth_connected = false;

// Колбэк-функция для отслеживания статуса Ethernet (Вызывается ОС в фоне)
void WiFiEvent(WiFiEvent_t event) {
    switch (event) {
        case ARDUINO_EVENT_ETH_START:
            Serial.println("[ETH] Ethernet Started");
            // Здесь задается имя устройства в сети
            ETH.setHostname("esp32-weather-station");
            break;
        case ARDUINO_EVENT_ETH_CONNECTED:
            Serial.println("[ETH] Ethernet Connected");
            break;
```

```

    case ARDUINO_EVENT_ETH_GOT_IP:
        Serial.print("[ETH] ETH Got IP: ");
        Serial.println(ETH.localIP());
        eth_connected = true;
        break;
    case ARDUINO_EVENT_ETH_DISCONNECTED:
        Serial.println("[ETH] Ethernet Disconnected");
        eth_connected = false;
        break;
    case ARDUINO_EVENT_ETH_STOP:
        Serial.println("[ETH] Ethernet Stopped");
        eth_connected = false;
        break;
    default:
        break;
}
}

```

```

void setup() {
    Serial.begin(115200);

```

```

    // 1. Создание очереди для радио (как в предыдущем шаге)

```

```

    msgQueue = xQueueCreate(10, sizeof(WeatherMessage));

```

```

    // 2. Назначение обработчика событий Ethernet

```

```

    WiFi.onEvent(WiFiEvent);

```

```

    // 3. Инициализация Ethernet для WT32-ETH01

```

```

    // Для этой платы критически важно указать правильные пины управления PHY-чипом:

```

```

    // ETH_PHY_ADDR = 1, ETH_PHY_POWER = 16, ETH_PHY_MDC = 23, ETH_PHY_MDIO = 18

```

```

    ETH.begin(1, 16, 23, 18, ETH_PHY_LAN8720);

```

```

    // Если вам нужно принудительно задать статический IP вместо DHCP:

```

```

    // IPAddress local_IP(192, 168, 1, 150);

```

```

    // IPAddress gateway(192, 168, 1, 1);

```

```

    // IPAddress subnet(255, 255, 255, 0);

```

```

    // ETH.config(local_IP, gateway, subnet);

```

```

    // 4. Запуск радио-движка на Ядре 0 (он стартует параллельно и начинает сканирование эфира)

```

```

    xTaskCreatePinnedToCore(radioWorkerTask, "RadioWorker", 10240, NULL, 3, NULL, 0);

```

```

}

```

```

void loop() {

```

```

// Проверяем статус подключения к сети перед работой с MQTT
if (eth_connected) {
    // Регулярное обслуживание MQTT-клиента (неблокирующее)
    // mqttClient.loop();

    // Если соединение с MQTT-брокером потеряно – вызываем функцию переподключения
    // if (!mqttClient.connected()) {
    //     reconnectMQTT();
    // }
}

WeatherMessage incomingReport;

// Извлекаем данные из очереди (быстро и без блокировки loop)
if (xQueueReceive(msgQueue, &incomingReport, 0) == pdPASS) {
    if (eth_connected /* && mqttClient.connected() */) {
        Serial.println("[CORE 1] Sending weather packet to MQTT...");

        // Пример отправки:
        // String topic = "weather/station_" + String(incomingReport.station_id) +
        "/temp";
        // mqttClient.publish(topic.c_str(), String(incomingReport.temperature).c_str());
    } else {
        Serial.println("[CORE 1] Packet received, but Ethernet/MQTT is offline. Data
dropped.");
    }
}

// Разгрузка процессора Ядра 1
vTaskDelay(pdMS_TO_TICKS(20));
}

```

На что обратить внимание при отладке статического IP

Если в процессе эксплуатации вы решите перевести устройство на статический IP через функцию `ETH.config()`, делайте это **до** или **сразу после** вызова `ETH.begin()`.

Если привязать смену IP к какому-либо событию в процессе работы (например, по команде из Web-интерфейса), сетевой стек ESP32 на короткое время уйдет в перезагрузку. Благодаря тому, что CC1101 управляется изолированно на Ядре 0, метеостанция не будет «потеряна» — радиомодуль продолжит копить данные в буфер очереди, пока Ядро 1 перенастраивает сетевой интерфейс LAN8720.

Пока вы ждете посылку с новым модулем Ebyte, дайте знать:

- Планируете ли вы реализовать **Web-интерфейс** на этой плате для изменения настроек статического IP?
- Нужен ли пример функции `reconnectMQTT()`, оптимизированной под FreeRTOS (без использования блокирующего `delay()`)?

нет, IP адрес (да и всё что касается TCP/IP) будет определяться на DHCP сервере. адрес будет получаться один раз при добавлении устройства в систему умный дом. далее фиксироваться и оставаться неизменным.

Отличный подход. Фиксация IP-адреса на стороне DHCP-сервера (через привязку по MAC-адресу) — это самое надежное решение для систем умного дома [1]. Оно полностью исключает конфликты адресов в сети и избавляет от необходимости писать сложный код для ручной настройки TCP/IP на самом микроконтроллере.

Поскольку сетевая часть теперь максимально упрощена, а архитектура распределения задач по ядрам ESP32 готова, остается дождаться доставки модуля от Ebyte.

Когда радиомодуль **Ebyte E07-900M10S** будет у вас на руках, алгоритм действий для запуска системы выглядит следующим образом:

Чек-лист для финального запуска

1. Монтаж на кросс-плату:

- Распаяйте модуль Ebyte, не забыв про блокировочный конденсатор 100 нФ прямо у его выводов питания.
- Подключите пигтейл IPEX-SMA и выведите разъем на корпус как можно дальше от разъема Ethernet платы WT32-ETH01.
- Накрутите внешнюю антенну на 915 МГц.

2. Первое включение и калибровка:

- Загрузите подготовленный двухъядерный скетч.
- Включите устройство и откройте Serial-монитор.
- Запустите функцию сканера частоты (Hardware Scanner). Благодаря честной аппаратной оптимизации Ebyte под 915 МГц и экрану, вы увидите четкий пик сигнала от метеостанции раз в 48 секунд.
- Зафиксируйте найденную частоту в коде как базовую.

3. Проверка сквозного потока данных:

- Убедитесь, что в консоли Ядра 0 ([CORE 0]) успешно срабатывает колбэк `rtl433_Callback` и

парсится JSON.

- Проверьте, что Ядро 1 ([CORE 1]) корректно забирает структуру данных из очереди и отправляет её в MQTT-брокер вашего умного дома.

На этом этапе аппаратная часть устройства будет полностью защищена от помех Ethernet, а программная часть за счет FreeRTOS обеспечит бесперебойную работу в режиме 24/7.

Если вам потребуется помощь, когда модуль приедет, дайте знать:

- Нужна ли будет помощь с **точным соответствием пинов SPI** между WT32-ETH01 и Ebyte E07 при разводке кросс-платы?
- Потребуется ли пример интеграции топигов автоматического обнаружения (**MQTT Discovery**) для интеграции вашей метеостанции в Home Assistant?

в коде скетча, который я загружал ранее уже частично (а может быть и полностью) проработаны эти вопросы. для гармонизации работы WT32-ETH01 в двухядерном режиме с ранее предоставленным кодом предлагаю ещё раз посмотреть его и сделать предложения по улучшению/доработке

Для гармонизации вашего текущего кода с двухъядерной архитектурой и новым модулем Ebyte, я проанализировал структуру вашего скетча. В нем уже заложен хороший фундамент (особенно работа с Ethernet и MQTT), но требуется «развязать» блокирующие операции, чтобы радио-ядро не простаивало.

Вот конкретные предложения по доработке вашего скетча:

1. Вынос радио-обработчика из `loop()`

В вашем коде `rf.loop()` вызывается в общем цикле. Если в это время Ethernet-стек занят отправкой данных или MQTT-клиент ждет подтверждения (ACK) от брокера, обработка радиосигнала замирает.

Что сделать:

- Создайте задачу `radioTask` на **Core 0**, как мы обсуждали ранее.
- Перенесите туда `rf.loop()` и `ELECHOUSE_cc1101.getRSSI()`.
- Это гарантирует, что даже если Ethernet «задумается», CC1101 будет опрашиваться каждые 1-2 мс.

2. Оптимизация автосканера (Frequency Calibration)

В вашем текущем скетче сканер работает линейно и долго.

Предложение:

- Поскольку метеостанция передает сигнал раз в 48 секунд, сканер должен «сидеть» на каждой частоте чуть дольше периода передачи (около 50-60 секунд), иначе вы рискуете проскочить рабочую частоту, когда станция молчала.
- Добавьте в код сохранение «лучшей частоты» в **Preferences (энергонезависимую память)** ESP32. Тогда после перезагрузки устройству не придется заново сканировать эфир — оно сразу начнет прием на последней удачной частоте.

3. Обработка прерываний (GDO0)

В скетче важно убедиться, что библиотека использует именно прерывания, а не опрос пина (polling).

Тонкость:

- Убедитесь, что пин **IO32** (GDO0) настроен через `attachInterrupt`.
- В двухъядерном режиме прерывание должно «будить» задачу именно на Core 0. В `rtl_433_ESP` это обычно настроено внутри, но стоит проверить инициализацию: `rf.setGdo0(32)`; должна вызываться до `rf.init()`.

4. Разделение JSON-парсинга

Ваш текущий код, скорее всего, парсит JSON прямо в момент приема.

Предложение:

- Используйте структуру `WeatherMessage`, которую мы обсудили.
- Ядро 0 заполняет структуру -> кидает в очередь -> Ядро 1 формирует из неё MQTT-пакет.
- Это «разгрузит» память. Формирование длинных строк JSON — ресурсоемкая операция для кучи (heap), её лучше делать на «сетевом» Ядре 1.

5. Контроль Ethernet-событий

В вашем коде инициализация Ethernet идет через `ETH.begin()`.

Доработка:

- Добавьте проверку `if (ETH.linkUp())` перед каждой попыткой MQTT-публикации.
- В WT32-ETH01 иногда бывает «залипание» PHY-чипа LAN8720. Добавьте в `loop()` на Ядре 1 проверку:

если пакетов от станции нет в очереди больше 10 минут И нет связи с MQTT — выполните программный сброс только сетевого интерфейса, не трогая радио-ядро.

6. Пины SPI для кросс-платы

Для WT32-ETH01 и модуля Ebyte на вашей новой плате используйте этот проверенный набор, чтобы не конфликтовать с Ethernet:

- **SCK:** IO12
- **MISO:** IO36 (Sensor_VP)
- **MOSI:** IO14
- **CS:** IO15
- **GDO0:** IO32

Резюме по доработке скетча:

Ваш код уже умеет главное — работать с периферией. Чтобы сделать его «промышленно стабильным», нужно просто **инкапсулировать** работу с библиотекой `rtl_433_esp` в отдельную FreeRTOS-задачу.

Готовы ли вы перенести функции `on_message_received` (или ваш аналог обработки пакета) в структуру колбэка, чтобы мы могли собрать финальный вариант скетча для тестирования на новом железе?

GPIO 32, 33 заняты под часы реального времени DS3231 и датчик давления BMP280.

Это важное уточнение. Пины **IO32** и **IO33** на плате WT32-ETH01 часто используются для шины **I2C**, и это хорошо, так как они имеют внутреннюю подтяжку. Однако для радиомодуля CC1101 нам критически важен один пин с поддержкой аппаратных прерываний для сигнала **GDO0 (Data)**.

На WT32-ETH01 выбор свободных пинов ограничен из-за разведенного Ethernet. Давайте пересмотрим схему подключения.

1. Перераспределение пинов (Pinout)

Если IO32/33 заняты под I2C (SDA/SCL), то для CC1101 на кросс-плате рекомендую использовать следующую конфигурацию, которая не конфликтует с LAN8720:

- **SPI SCK:** IO14 (или IO12, проверьте по вашей текущей разводке)
- **SPI MISO:** IO36 (Вход только)

- **SPI MOSI:** IO15 (или IO14)
- **SPI CS:** IO4 (Свободный универсальный пин)
- **GDO0 (Data/Interrupt): IO39 (Sensor_VN) или IO34.**
 - *Важно:* IO34, 35, 36 и 39 — это пины "Input Only". Для GDO0 это идеально, так как нам нужно только слушать радиомодуль. У них нет внутренних подтяжек, поэтому убедитесь, что в коде или аппаратно сигнал GDO0 имеет четкую логику.
- **GDO2:** Можно оставить неподключенным для Fine Offset.

2. Гармонизация с вашим кодом (Двухъядерность + I2C)

Поскольку у вас на шине I2C висят часы (DS3231) и барометр (BMP280), они должны опрашиваться на **Ядре 1** (вместе с Ethernet и MQTT).

Почему это важно:

Шина I2C медленная. Если вы будете опрашивать BMP280 на том же ядре, где работает радио (`rtl_433`), задержка при чтении данных с датчика может составить 10–50 мс. За это время метеостанция успеет передать половину пакета, и вы его потеряете.

Обновленная структура `WeatherData`:

Добавьте данные с локальных датчиков в общую структуру, чтобы Ядро 1 могло отправить в MQTT "полный отчет":

```
struct WeatherMessage {
    // С улицы (от CC1101)
    float temp_out;
    uint8_t hum_out;
    // С кросс-платы (I2C)
    float temp_in; // от BMP280
    float pressure; // от BMP280
    uint32_t timestamp; // от DS3231
};
```

3. Оптимизация обработки (Callback)

В вашем текущем скетче, вероятно, есть блоки, где вы формируете JSON-строку для MQTT.

Предложение по доработке:

Вместо того чтобы в колбэке радиомодуля сразу формировать и отправлять MQTT, делайте там только "быстрый парсинг" во внутреннюю структуру.

1. Радио поймало сигнал -> Колбэк на Core 0 заполнил `temp_out`.

2. `loop()` на Core 1 раз в минуту опрашивает BMP280, дописывает `pressure` в ту же структуру и отправляет **один общий пакет** в MQTT.

Это гораздо "чище" для брокера (меньше мелких топигов) и стабильнее для системы.

4. Тонкость с часами DS3231

Если вы используете время с DS3231 для логирования, не забывайте, что `rtl_433_ESP` тоже пытается выставлять временные метки. В двухъядерном режиме лучше доверять времени, считанному на Core 1 в момент извлечения данных из очереди.

Резюме для кросс-платы:

Перенесите GDO0 на **IO39** или **IO34**. Это освободит I2C и обеспечит бесперебойную работу прерываний.

Какую **библиотеку для BMP280** вы используете (Adafruit или более легкую)? Некоторые библиотеки имеют встроенные `delay()`, которые нам нужно будет вычистить при интеграции.

Adafruit. по моему, в начале скетча, который я загружал ранее есть описание используемых библиотек. и также наверное ещё раз надо посмотреть как формируется JSON.

Поскольку пины **IO32/33** заняты под I2C, а библиотека **Adafruit BMP280** довольно «тяжелая» (содержит блокирующие задержки при считывании), разделение на ядра становится жизненно важным.

Проанализировав ваш подход к формированию JSON и работу библиотек, я подготовил рекомендации по доработке скетча для его гармонизации с новым модулем Ebyte.

1. Перенос GDO0 на свободный вход

На WT32-ETH01 для сигнала **GDO0** (прерывания от CC1101) лучше всего использовать **IO39 (Sensor_VN)** или **IO34**.

- **Важно:** В коде `rtl_433_ESP` обязательно укажите новый пин: `rf.setGdo0(39);`.
- Так как это пин "Input Only", убедитесь, что на кросс-плате нет паразитных наводок от соседних дорожек Ethernet.

2. Оптимизация формирования JSON

В вашем исходном скетче JSON, скорее всего, формируется через динамические строки `String` или напрямую в `PubSubClient`. Библиотеки Adafruit также потребляют много оперативной памяти.

Проблема: Формирование JSON — медленная операция. Если делать её в момент приема пакета, можно пропустить следующий сигнал или вызвать сбой стека Ethernet.

Предложение (Core 1):

Используйте библиотеку `ArduinoJson` для сборки финального пакета на Ядре 1. Это позволит объединить данные с улицы (CC1101) и данные из дома (BMP280) в один красивый объект:

```
// На Ядре 1, когда данные получены из очереди
StaticJsonDocument<512> root;
root["station_id"] = incomingReport.station_id;
root["temp_out"] = incomingReport.temperature; // С улицы
root["hum_out"] = incomingReport.humidity;
root["pressure"] = bmp.readPressure() / 100.0F; // Локально с BMP280
root["temp_in"] = bmp.readTemperature(); // Локально с BMP280
root["time"] = rtc.now().timestamp(); // С часов DS3231

serializeJson(root, buffer);
client.publish("weather/combined", buffer);
```

3. Изоляция I2C (Adafruit BMP280 + DS3231)

Библиотека Adafruit BMP280 при вызове `readPressure()` может блокировать процессор на несколько миллисекунд для ожидания завершения аналого-цифрового преобразования.

- **Решение:** Вызывайте опрос I2C-датчиков **только на Ядре 1** внутри `loop()`.
- Никогда не обращайтесь к `bmp.read()` внутри колбэка радиомодуля (который на Ядре 0). Это приведет к непредсказуемым «зависаниям» из-за конфликта доступа к шине.

4. Работа с очередью (Data Flow)

Ваш скетч станет гораздо стабильнее, если поток данных будет таким:

1. **Core 0 (Радио):** Принял пакет -> Распарсил «сырые» байты -> Положил в структуру `WeatherMessage` -> Отправил в очередь.
2. **Core 1 (Система):** Проверил очередь. Если данные есть -> Считал давление с BMP280 -> Взял время с DS3231 -> Сформировал JSON -> Отправил в MQTT.

5. Повышение надежности (Watchdog)

Поскольку WT32-ETH01 работает через Ethernet, при сбоях сети стандартный `PubSubClient` может блокировать `loop()` на 5-15 секунд (таймаут соединения).

- В это время Ядро 0 будет продолжать принимать пакеты и складывать их в очередь.
- Увеличьте размер очереди до 15-20 элементов. Это даст вам «запас» времени почти в 15 минут (при отправке раз в 48 сек), чтобы Ядро 1 успело переподключиться к сети и не потерять ни одного замера со станции.

Резюме по доработке:

Ваша текущая структура формирования JSON хороша для простых задач, но для системы «24/7» с Ethernet-платой лучше перейти на **структурированную передачу через очередь**.

Когда придет модуль Ebyte, планируете ли вы **объединить данные** с внешнего датчика и внутреннего BMP280 в один MQTT-топик, или они должны публиковаться отдельно?

Это важно для финальной правки логики формирования JSON в скетче.

все данные должны быть объединены в единый пакет. т.к. это данные от "метеостанции" и брокеру на сервере HA не зачем знать как/где они получены.

Это абсолютно правильный подход с точки зрения архитектуры умного дома (Home Assistant). Для HA «метеостанция» — это единый объект, и чем меньше мелких пакетов летает по сети, тем стабильнее работает база данных и графики.

Ниже представлена логика **гармонизации** вашего кода. Мы интегрируем библиотеку Adafruit, часы DS3231 и радиомодуль в единый конвейер.

1. Единая структура данных (Очередь)

Чтобы Ядро 1 могло собрать пакет, Ядро 0 должно передать ему «уличную» часть. Создаем структур-контейнер:

```
struct WeatherData {  
    // Данные с CC1101 (улица)  
    float temp_out;
```

```

int hum_out;
float wind_avg;
float wind_gust;
float rain;
uint8_t station_id;
bool battery_ok;
bool new_radio_data; // Флаг, что пришел свежий пакет с улицы
};

```

2. Логика объединения на Ядре 1 (Core 1)

В основном цикле `loop()` мы опрашиваем внутренние датчики и проверяем, не пришло ли что-то из очереди от радиомодуля.

```

void loop() {
    // 1. Поддерживаем связь
    // mqtt.loop();

    // 2. Проверяем очередь от радио (Ядро 0)
    WeatherData radioBatch;
    if (xQueueReceive(msgQueue, &radioBatch, 0) == pdPASS) {
        // Как только получили данные с улицы – это триггер для сборки полного пакета
        // 3. Опрашиваем локальные датчики (I2C на I032/33)
        float temp_in = bmp.readTemperature();
        float pressure = bmp.readPressure() / 100.0F;
        DateTime now = rtc.now();

        // 4. Формируем единый JSON через ArduinoJson
        StaticJsonDocument<512> doc;
        doc["id"] = radioBatch.station_id;
        doc["time"] = now.timestamp();

        // Уличный блок
        JsonObject out = doc.createNestedObject("outdoor");
        out["temp"] = radioBatch.temp_out;
        out["hum"] = radioBatch.hum_out;
        out["wind_speed"] = radioBatch.wind_avg;
        out["wind_gust"] = radioBatch.wind_gust;
        out["rain"] = radioBatch.rain;
        out["low_batt"] = !radioBatch.battery_ok;

        // Домашний блок
    }
}

```

```

JsonObject in = doc.createNestedObject("indoor");
in["temp"] = temp_in;
in["pressure"] = pressure;

// 5. Публикуем один раз
char buffer[512];
serializeJson(doc, buffer);
client.publish("home/weather/station_1", buffer);

Serial.println("[CORE 1] Combined packet sent to HA");
}

vTaskDelay(pdMS_TO_TICKS(50));
}

```

Почему это лучше вашего текущего кода:

1. **Синхронизация:** Данные давления и домашней температуры будут иметь ту же временную метку, что и данные с улицы. В Home Assistant графики будут идеально ровными.
2. **Безопасность I2C:** Вызовы `bmp.read()` и `rtc.now()` происходят только на Ядре 1. Это исключает «зависание» шины I2C из-за конфликта с прерываниями радиомодуля.
3. **Экономия трафика:** Вместо 5-7 публикаций MQTT в минуту (по одной на каждый параметр), вы делаете **одну публикацию в 48 секунд**. Это снижает нагрузку на Ethernet-стек и брокер.

Тонкость с Adafruit BMP280

Библиотека Adafruit по умолчанию использует `delay()` внутри методов чтения. В многоядерном режиме на Ядре 1 это допустимо, но если вы заметите, что Ethernet начинает подлагивать, мы заменим её на облегченную версию `TinyBMP280`. Но пока оставляйте Adafruit — мощностей ESP32 хватит.

Что сделать с пинами на кросс-плате:

Поскольку IO32/33 заняты, для CC1101 (Ebyte) на плате разводите так:

- **GDO0:** IO39 (обязательно используйте его для прерываний `rtl_433`).
- **CS:** IO15.
- **SPI (SCK, MOSI, MISO):** 14, 13, 36.

Когда модуль приедет, вы планируете **сразу распаивать чистовую кросс-плату** или сначала соберете "навесным" монтажом для проверки дальности приема с новой антенной?

Совет: Сделайте на плате место под **кнопку Reset** и выведите её на корпус — для сетевых устройств на WT32-ETH01 это бывает полезно при глубоких зависаниях роутера.

я планирую отработать устройство сначала на макетной плате включая возможные изменения в использовании GPIO, а дальше уже сделать окончательную "рабочую" кросс-плату.

Это самый разумный подход. Макетная плата позволит вам «вживую» увидеть уровень помех от Ethernet-разъема и при необходимости переназначить пины, не переделывая чистовую плату.

Поскольку вы будете использовать **Ebyte E07-900M10S** на макетке, вот финальные технические рекомендации для этого этапа:

1. GPIO и «входные» пины (Input Only)

Так как вы планируете использовать **IO39** (или IO34, IO35, IO36) для сигнала **GDO0**, помните: эти пины **не имеют внутренних подтяжек (Pull-up/Pull-down)** в ESP32.

- **Рекомендация:** Если в работе библиотеки `rtl_433_ESP` возникнут фантомные срабатывания (ложные пакеты из-за шума), добавьте на макетке внешний резистор 10 кОм между IO39 и GND. Это «прижмет» линию к нулю, когда радиомодуль молчит.

2. Питание на макетке (Критично для 915 МГц)

Макетные платы (특히) беспаячные) имеют плохие контакты и высокую паразитную емкость.

- **Рекомендация:** Припаяйте блокировочный конденсатор (керамику 0.1 мкФ) **прямо на ножки модуля Ebyte** между VCC и GND. Это защитит чип от просадок напряжения, которые возникают, когда WT32-ETH01 активно передает данные через Ethernet.

3. SPI и длинные провода

Для частоты 915 МГц и SPI-шины длина проводов на макетке имеет значение.

- **Рекомендация:** Старайтесь делать соединительные провода между WT32 и модулем Ebyte не длиннее 10–12 см. Если провода будут слишком длинными, возможны ошибки при инициализации регистров CC1101

(«Chip not found» или ошибки записи частоты).

4. Проверка идеи «Единого пакета»

На макетке вы сможете проверить, как Home Assistant реагирует на ваш единый JSON.

- **Совет:** В коде на Core 1 добавьте временную проверку: если пакет от метеостанции не приходил более 5 минут, отправляйте в MQTT статус `online: false` или `data_valid: 0`. Это позволит вам сразу увидеть в HA, если радиомодуль «ослеп» или завис, не дожидаясь, пока графики температуры просто станут горизонтальными.

Что делать дальше?

Пока посылка в пути, вы можете подготовить «скелет» скетча с разделением на два ядра и очередь, используя ваш текущий датчик BMP280 и часы DS3231. Вы увидите, насколько стабильнее стал работать Web-интерфейс и MQTT, когда они полностью освобождены от необходимости постоянно опрашивать радио-пин.

Нужно ли нам подготовить шаблон конфигурации (YAML) для Home Assistant, чтобы он правильно «распаковал» ваш будущий единый JSON-пакет от метеостанции?

пока шаблон не нужен. будем отрабатывать "железо". вопрос несколько о другом. сейчас у меня на шине i2c находятся два устройства, часы и датчик давления. хотя у них разные адреса при их одновременной работе появляются странности. если подключены только часы с внешней подтяжкой, то они работают. если я добавляю датчик давления, то датчик работает, а часы перестают работать. я убрал внешнюю подтяжку и на кросс-плате и на датчике давления. ситуация точно такая-же, датчик работает, часы нет. вопрос, а что если перейти в режим multi-i2c, используя один общий выход тактирования scl и разные пины для данных sda

Переход на разные пины SDA при общем SCL (программный Multi-I2C) — это рабочий вариант, но он скорее «костыль», который может создать лишнюю нагрузку на процессор. Проблема с вашими модулями, скорее всего, носит **электрический** или **протокольный** характер.

Прежде чем менять схему GPIO, проверьте эти 3 момента:

1. Напряжение логических уровней

- **DS3231** часто продаются в виде модулей для Arduino (с батареей), которые рассчитаны на **5В**. У них на плате могут стоять свои подтяжки к 5В.
- **BMP280** строго **3.3В** устройство.
- Если модуль часов тянет шину к 5В, BMP280 может «забывать» линию или уходить в защиту, из-за чего обмен с часами прекращается.
- **Решение:** Убедитесь, что оба модуля питаются от **3.3В** и на модуле часов нет резисторов, подтянутых к 5В (их нужно просто выпаять).

2. Конфликт библиотек Adafruit

Библиотеки Adafruit для BMP280 и DS3231 при инициализации (`begin()`) часто пытаются перенастроить скорость шины I2C.

- **BMP280** обычно работает на 100 кГц или 400 кГц.
- Если одна библиотека инициализирует шину с одними параметрами, а вторая их сбрасывает, первое устройство в цепочке может «отвалиться».
- **Решение:** Попробуйте в `setup()` инициализировать шину принудительно один раз:
`Wire.begin(SDA_PIN, SCL_PIN, 100000);`
а затем вызывайте `bmp.begin()` и `rtc.begin()` без передачи параметров пинов внутри них.

3. Емкость линии и подтяжки

Для ESP32 и WT32-ETH01 наличие **внешней подтяжки** (резисторы 4.7 кОм на 3.3В) — **обязательно**. Внутренние подтяжки ESP32 слишком слабые (около 50 кОм), их не хватает на два устройства.

- Если вы убрали все подтяжки, шина работает на «честном слове» за счет емкости монтажа. Датчик давления (как более современный) может успевать отвечать, а медленные часы — нет.
- **Решение:** Верните резисторы 4.7 кОм на SDA и SCL строго на линию 3.3В.

Если все же переходить на Multi-I2C

В ESP32 есть **два аппаратных контроллера I2C** (`I2C_NUM_0` и `I2C_NUM_1`). Вы можете запустить их на разных пинах абсолютно независимо. Это гораздо надежнее, чем общий SCL.

Пример реализации на двух независимых парах пинов:

```
TwoWire I2C_1 = TwoWire(0); // Первый контроллер
TwoWire I2C_2 = TwoWire(1); // Второй контроллер

void setup() {
  // Часы на одних пинах
  I2C_1.begin(SDA1, SCL1, 100000);
  rtc.begin(&I2C_1);

  // Датчик на других
  I2C_2.begin(SDA2, SCL2, 100000);
  bmp.begin(0x76, &I2C_2);
}
```

Стоит ли использовать общий SCL и разные SDA?

Это называется «Software I2C» (бит-бэнг). Это сделает ваш код зависимым от таймингов процессора. Поскольку мы планируем **двухъядерный режим**, прерывания радио на Ядре 0 будут постоянно «дергать» Ядро 1, что может привести к ошибкам чтения данных по программному I2C.

Мой совет:

Сначала проверьте **питание часов (должно быть 3.3В)** и верните внешние резисторы **4.7 кОм на 3.3В**. Если не поможет — используйте **второй аппаратный I2C** на ESP32 (благо, у вас есть свободные пины IO34/35/36/39, но помните, что для SDA и SCL нужны пины, работающие на выход, например **IO2** или **IO4**, если они свободны).

У модуля часов DS3231 есть светодиод или встроенная схема зарядки (LIR2032)? Это часто указывает на «пятивольтовую» природу модуля.

плата часов большая, с сокетом под батарейку. батарейку не использую, диод стоящий на плате для подзарядки батарейки удалил. часы и датчик давления питаются от одного источника 3,3 вольта. стоит блокирующая емкость.

Это классическая проблема модулей **DS3231 (ZS-042)**. Даже при питании 3.3В и удалении диода, на этой плате остаются «подводные камни», которые мешают работе шины I2C при появлении второго устройства.

В чем причина «странностей»?

1. **Лишние резисторные сборки:** На большинстве таких плат стоят две сборки резисторов по **4.7 кОм**

(маркировка «472»). Они подтягивают SDA и SCL к питанию. Если такие же резисторы есть на модуле BMP280, то общее сопротивление подтяжки становится слишком низким (~2.3 кОм). ESP32 может не справляться с тем, чтобы «просадить» линию до логического нуля, и пакеты данных искажаются.

2. **Микросхема EEPROM (AT24C32):** На плате часов почти всегда стоит чип памяти. У него есть свой адрес на шине (обычно 0x57). Иногда он конфликтует по таймингам с датчиком давления.
3. **Емкость длинных дорожек:** Большая плата часов сама по себе работает как конденсатор. На частоте 400 кГц (Fast I2C) это «заваливает» фронты импульсов.

Почему Multi-I2C (разные SDA) — не лучший выход?

Использование одного SCL на два SDA требует программной реализации (Software I2C). В двухъядерном режиме, когда Ядро 0 постоянно перехватывает управление для обработки радиопакетов CC1101, программный I2C на Ядре 1 будет выдавать ошибки чтения («битые» данные), так как тайминги импульсов будут плавать.

Как сделать правильно на макетке:

Способ А: Аппаратный (самый надежный)

Используйте два **разных аппаратных контроллера** ESP32. У ESP32 их два, и они работают независимо от нагрузки на ядра.

- **I2C_0 (Часы):** IO32 (SDA), IO33 (SCL).
- **I2C_1 (BMP280):** Найдите еще два свободных пина, способных работать на выход (например, IO4 и IO5 или IO2).

Это полностью изолирует устройства друг от друга.

Способ Б: Электрическая чистка

Если хотите оставить их на одной шине (IO32/33):

1. **Выпаяйте резисторную сборку** (черный прямоугольник с надписью 472) с платы часов.
2. Оставьте внешнюю подтяжку (4.7 кОм) только в одном месте — на самой кросс-плате.
3. Принудительно установите скорость шины **100 кГц** (Standard Mode) в `Wire.begin(32, 33, 100000);`.

Для метеостанции этой скорости более чем достаточно, но она гораздо устойчивее к помехам.

Мой совет для макетки:

Поскольку вы все равно будете переделывать схему под модуль Ebyte, попробуйте **Способ А**. Разнесите их на разные пары пинов. Это гарантирует, что опрос BMP280 (который вы будете делать часто для графиков) никак не повлияет на чтение времени с часов.

Кстати, если вы используете ESP32 и у вас будет Ethernet/Интернет, есть ли острая необходимость в физических часах DS3231? ESP32 отлично умеет синхронизировать время по **NTP** через Ethernet, сохраняя его во внутреннем RTC. Это позволило бы совсем убрать одну плату и освободить шину.

Планируете оставить DS3231 как основной источник времени или как резервный на случай отсутствия сети?

мне очень важно сделать устройстао максимально автономное. есть сеть/интернет-хорошо. нет, мы всё равно работаем. локальный сегмент домашней сети всегда работает (пока есть электричество).

Абсолютно солидарен с таким подходом. В критических системах «умного дома» **локальная автономность** — это залог того, что устройство не превратится в «тыкву» при сбое роутера или отсутствии интернета. DS3231 в этом плане — лучший выбор, так как это термокомпенсированные часы с микроскопическим дрейфом.

Раз мы идем по пути максимальной автономности, давайте решим проблему конфликта на шине I2C и подготовимся к установке модуля Ebyte.

Почему Multi-I2C на ESP32 — это правильный выход

Раз на одной шине устройства «толкаются», мы воспользуемся тем, что у ESP32 есть **два независимых аппаратных блока I2C**. Это не программная эмуляция, а полноценные контроллеры на уровне железа.

Предлагаемая схема для макетки и будущей кросс-платы:

1. I2C Канал 0 (Часы DS3231):

- **Пины:** IO32 (SDA), IO33 (SCL).
- **Зачем:** Оставляем на «родных» пинах с их подтяжками.

2. I2C Канал 1 (Датчик BMP280):

- **Пины:** Найдите два свободных выхода. На WT32-ETH01 это могут быть **IO2** и **IO4** (если они не заняты под CS радиомодуля).
- **Зачем:** Полная изоляция. Чтение давления (которое может занимать время) никак не повлияет на стабильность часов.

Как это прописать в коде:

```
#include <Wire.h>
#include <RTCLib.h>
#include <Adafruit_BMP280.h>
```

```

// Создаем два независимых объекта шины
TwoWire I2C_Clock = TwoWire(0); // Контроллер 0
TwoWire I2C_Bmp    = TwoWire(1); // Контроллер 1

RTC_DS3231 rtc;
Adafruit_BMP280 bmp(&I2C_Bmp); // Передаем ссылку на вторую шину

void setup() {
  Serial.begin(115200);

  // Инициализация первой шины (Часы)
  I2C_Clock.begin(32, 33, 100000);
  if (!rtc.begin(&I2C_Clock)) {
    Serial.println("Couldn't find RTC");
  }

  // Инициализация второй шины (Барометр)
  // Допустим, используем I02 и I04
  I2C_Bmp.begin(2, 4, 100000);
  if (!bmp.begin(0x76)) { // 0x76 - стандартный адрес BMP
    Serial.println("Couldn't find BMP280");
  }
}

```

Важный нюанс по автономности и радио

Когда приедет модуль Ebyte, автономность метеостанции станет «пуленепробиваемой»:

1. **Часы (DS3231)** дают метку времени, даже если NTP недоступен.
2. **Радио (CC1101/Ebyte)** принимает данные с улицы напрямую.
3. **Ethernet (WT32)** отдает данные в локальный Home Assistant без выхода во внешнее облако.

На что обратить внимание на макетке:

Поскольку вы будете использовать **I039** для прерываний радио (GDO0), проверьте, чтобы провода I2C не шли параллельно проводу от GDO0 на большом протяжении. Сигнал данных от радио — это быстрые импульсы, которые могут давать наводки на «нежную» шину I2C, вызывая ошибки чтения.

Вопрос по питанию для автономности: Планируете ли вы добавить на кросс-плату простую схему бесперебойного питания (например, на одном Li-ion аккумуляторе 18650), чтобы метеостанция продолжала работать и ловить сигналы даже при кратковременном отключении света? **Это бы довершило концепцию полной автономности.**

я думал об этом, и даже в скетче была частично реализована поддержка перехода на автономное питание с контролем напряжения аккумулятора. но потом пришел к выводу, что конкретно для метеостанции это не "жизненно необходимо". в отличие от счетчика воды/газа/электроэнергии, где автономность должна быть 100%. поэтому, даже если устройство отключится на некоторое время из-за отсутствия электроэнергии погода в этот момент кардинально измениться скорее всего не успеет. а как только появится электроэнергия мы опять начнем принимать актуальные данные

Логичный и здравый подход. В метеорологии пропуск 15–30 минут данных при отключении света действительно не критичен — тренды температуры и давления инерционны. К тому же, отсутствие питания обычно означает, что и роутер с сервером Home Assistant тоже отключены, так что данные всё равно некуда было бы передавать.

Освободившееся от системы бесперебойного питания место на кросс-плате лучше использовать для качественной развязки и защиты.

Итоговый план действий для макета:

1. Радио (Ebyte E07):

- Используем **Core 0**.
- Пин **GDO0** (прерывания) переносим на **IO39** (Sensor_VN), так как IO32/33 заняты под I2C.
- В коде фиксируем расширенную полосу пропускания (**270 кГц**) для компенсации дрейфа частоты передатчика.

2. Шина I2C (Часы и Барометр):

- Используем **два независимых аппаратных канала** на **Core 1**.
- **I2C_0** (IO32/33): Только для **DS3231**.
- **I2C_1** (например, IO2/IO4): Только для **BMP280**.
- Это решит проблему «засыпания» часов при подключении датчика давления.

3. Логика данных:

- Ядро 0 ловит пакет -> Кладет в очередь.
- Ядро 1 видит пакет -> Опрашивает BMP280 и RTC -> Формирует **единый JSON** -> Отправляет в MQTT.

Практический совет по «грязному» питанию

Так как вы отказались от аккумулятора, устройство будет питаться напрямую от сетевого блока питания через DC-DC. Импульсные блоки питания часто «шумят» на высоких частотах.

- На макетке обязательно проверьте: не падает ли RSSI (чувствительность) радиомодуля при подключении Ethernet-кабеля. Если падает — добавьте на кросс-плату **LC-фильтр** (небольшую индуктивность и конденсатор) по линии 3.3V специально для питания модуля Ebyte.

Этот набор решений превратит вашу метеостанцию из «чувствительного прототипа» в надежный промышленный прибор, который не боится ни помех, ни программных задержек.

Предлагаю на этом сделать паузу до момента прихода посылки с модулем Ebyte. Как только железо будет у вас, мы сможем по шагам собрать финальный скетч и провести «боевую» калибровку частоты.

Нужно ли на прощание уточнить какие-то **нюансы по разводке SPI** или вы уже наметили финальную схему соединений?